# Transistors to transformers

## Come abbiamo insegnato alla sabbia a pensare

Luca Miglioli

# A journey thru the stack

## Suddividiamo in vari livelli di astrazione gli step per arrivare a gpt

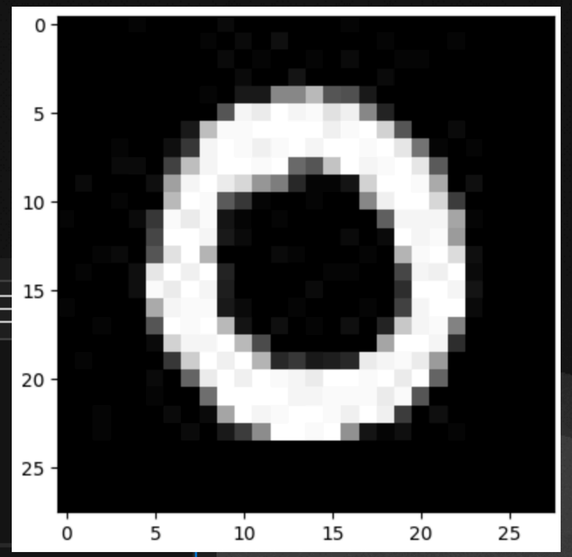
- Large scale training and inference
- Transformer architecture
- Linear layers
- Binary classification ← We start here
- Floating point ops
- Transistors and Gates
- Thinking in binary

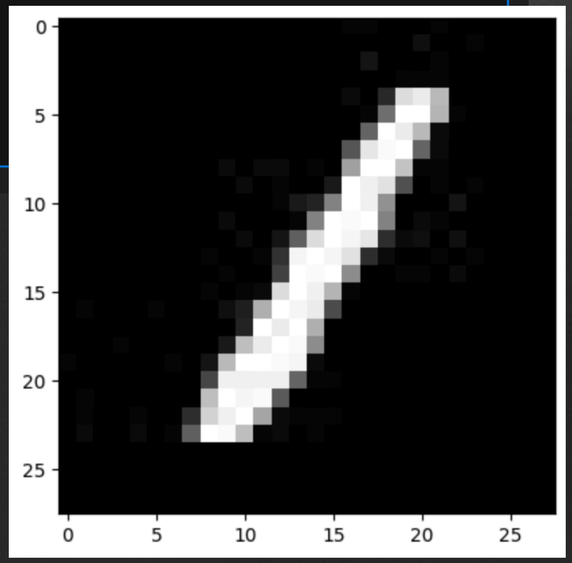
# Binary classification

## Understanding the fundamental operation in ML

```
images = torch.load('datasets/mnist_images.pt')/255
labels = torch.load('datasets/mnist_labels.pt')

print(labels[0])
plt.imshow(images[0])
plt.show()
print(labels[4000])
plt.imshow(images[4000])
plt.show()
```



# Binary classification

## Understanding the fundamental operation in ML

```python
# Parameters
m = torch.zeros(784).requires_grad_()
q = torch.zeros(1).requires_grad_()

# Training hyperparameters
epochs = 50
lr = 1e-3 #  1*10^-3 = 0.001

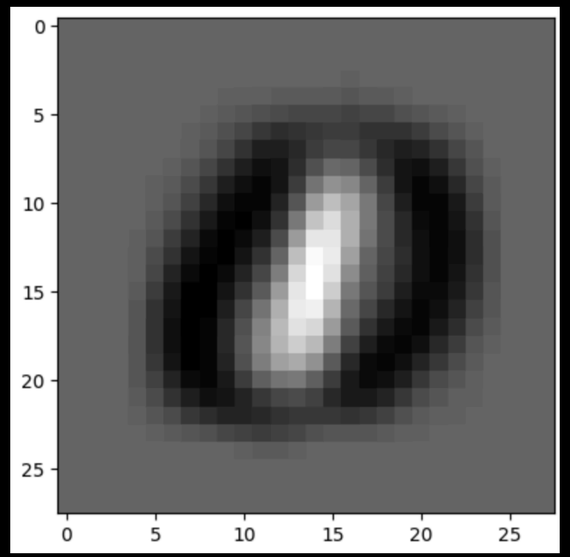
# Training loop
for epoch in range(epochs):
    # Forward pass
    # How shapes work in pytorch matrix multiplications: [8000, 784] @ [784] = [8000]
    pred = F.sigmoid(images @ m + q)  # formula for perceptron with sigmoid activation
    loss = ((pred - labels)**2).mean()

    # Backward pass
    loss.backward()
    with torch.no_grad():
        m -= lr * m.grad
        q -= lr * q.grad
        m.grad = None
        q.grad = None

    # Print loss every 10% of epochs
    if epoch % (epochs//10) == 0:
        print(f'Epoch {epoch}/{epochs} - Loss: {loss.item()} - Accuracy: {round(((pred.round() ==
```

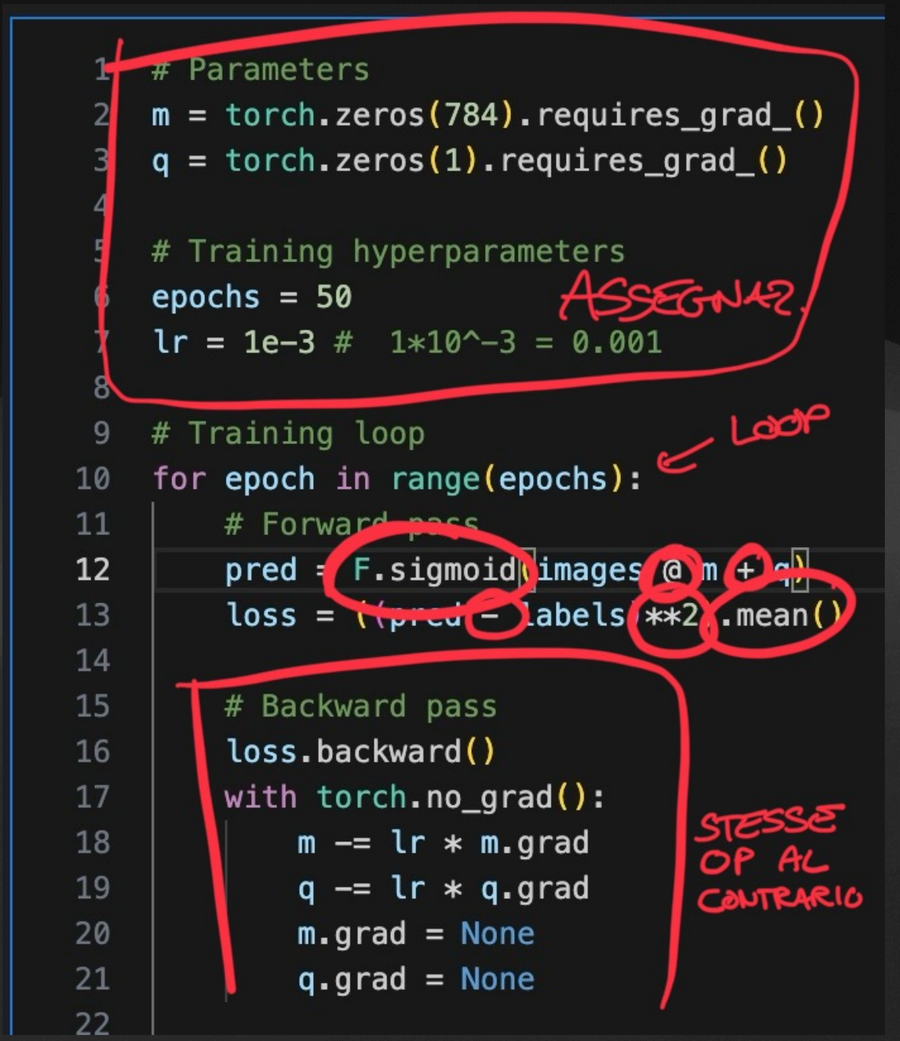
Python

```
Epoch 0/50 - Loss: 0.25 - Accuracy: 50%
Epoch 5/50 - Loss: 0.25 - Accuracy: 98%
Epoch 10/50 - Loss: 0.24 - Accuracy: 98%
Epoch 15/50 - Loss: 0.24 - Accuracy: 98%
Epoch 20/50 - Loss: 0.23 - Accuracy: 98%
Epoch 25/50 - Loss: 0.23 - Accuracy: 98%
Epoch 30/50 - Loss: 0.23 - Accuracy: 98%
Epoch 35/50 - Loss: 0.22 - Accuracy: 98%
Epoch 40/50 - Loss: 0.22 - Accuracy: 98%
Epoch 45/50 - Loss: 0.22 - Accuracy: 99%
```



Operazioni utilizzate:

Sigmoid
@
+
-
**2
Mean



## Operazioni utilizzate:

Sigmoid $\frac{1}{1+e^{-x}}$ ← DIV, SOMMA, EXP

@ ← SOMMA DI PRODOTTI

+

- ← SOMMA INVERSA

**2 ← PRODOTTO CON SE STESSO

Mean ← SOMMA E DIV

Riassumendo ci serve saper fare:

Somma, prodotto, divisione, esponenziale

# Transistors and Gates

## At the very bottom, there’s no math, no code, no AI — just a switch.

- P and N transistors

- Large scale training and inference
- Transformer architecture
- Linear layers
- Binary classification
- Floating point ops
- Adders and multipliers
- Transistors and Gates ← We are here

# Transistors and Gates

## Our unit of scale

- P + N transistors to make NOT gates
- 2P + 2N transistors to make a NAND gates

# Counting with bits

0, 1, 10, 11, 100, 101, 110, 111

# Adding up int4 numbers

## In column operations

# Demo of basic logic

## Half adder

# Floating point

# Adding and multiplying FP8

# Matrix Multiplication

**Multiply-Add matrix 1 rows with matrix 2 columns.**

Number of operations is proportional to:

- Rows in matrix 1
- Columns in matrix 2
- Length of rows(1) or columns(2)

(A, B) @ (B, C)

Complexity: A*B*C

(N, N) @ (N, N)

Complexity: N^3

Space: 2*N^2

# Matrix Multiplication

## Practical implications

(N, N) @ (N, N)

Complexity: N^3

Space: 2*N^2

Small matrices -> Bandwidth bound

Small matrices -> Compute bound